

National University of Technology

Artificial Neural Network Lab- CS3512

LAB Submitted to:

Maj. Rooshan Saleem

Submitted By:

Muhammad Abdullah

Reg. No: **F22607010**
Batch: **Fall AI 2022**
Lab : **06**

Q.NO.1:

```
import tensorflow as tf

data = tf.keras.datasets.mnist.load_data()

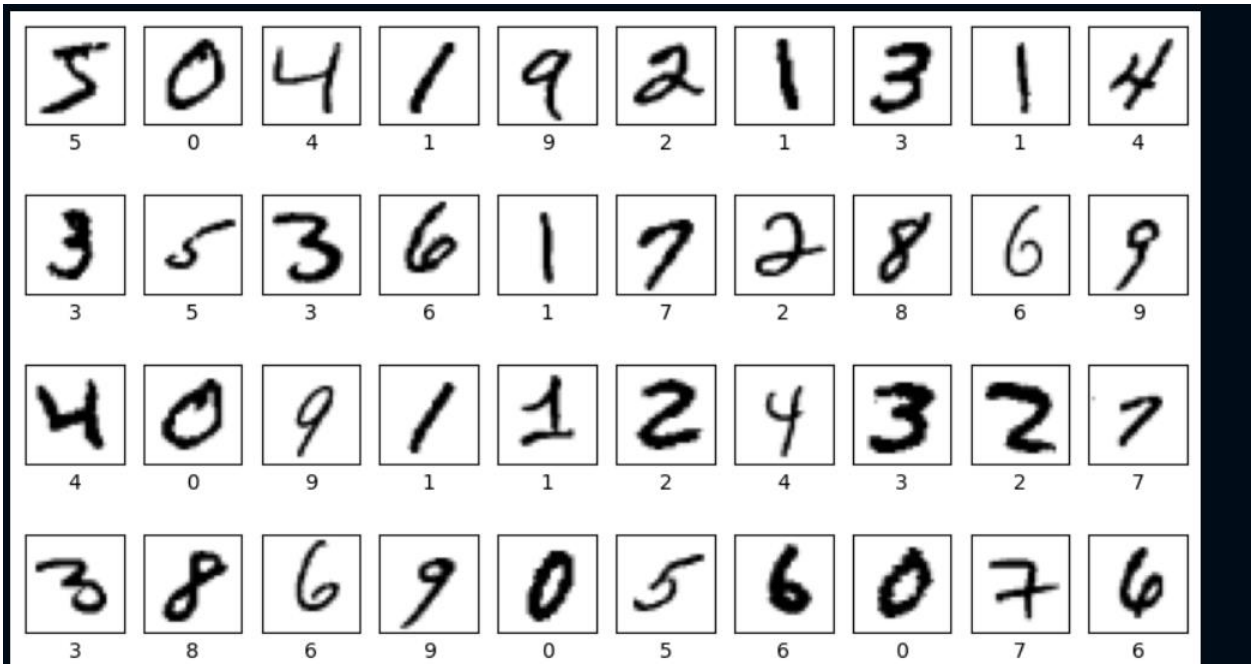
(x_train, y_train), (x_test, y_test) = data
print("X Train Shape: ",x_train.shape)
print("Y Train Shape: ",y_train.shape)
print("X Test Shape: ",x_test.shape)
print("Y Test Shape: ",y_test.shape)
```

```
X Train Shape:  (60000, 28, 28)
Y Train Shape:  (60000,)
X Test Shape:  (10000, 28, 28)
Y Test Shape:  (10000,)
```

```
import matplotlib.pyplot as plt
import numpy as np


plt.figure(figsize=(10,10))

for i in range(70):
    plt.subplot(7,10,i+1)
    plt.xticks([])
    plt.yticks([])
    plt.grid(True)
    plt.imshow(x_train[i],cmap=plt.cm.binary)
    plt.xlabel(y_train[i])
plt.show()
```




```
train_images , test_images = x_train / 255.0 , x_test / 255.0
train_images = train_images.reshape(train_images.shape[0],28,28,1)
test_images = test_images.reshape(test_images.shape[0],28,28,1)

print("Train Images Shape: ",train_images.shape)
print("Test Images Shape: ",test_images.shape)
```

```
Train Images Shape:  (60000, 28, 28, 1)
Test Images Shape:  (10000, 28, 28, 1)
```

```
from keras.models import Sequential
from keras.layers import Dense, Conv2D, Flatten, MaxPooling2D, Dropout
from keras.callbacks import EarlyStopping
from keras.optimizers import Adam

model = Sequential([
    Conv2D(filters=32, kernel_size=(3,3), activation='relu',
input_shape=(28,28,1)),
    MaxPooling2D(pool_size=(2,2)),
    Conv2D(filters=64, kernel_size=(3,3), activation='relu'),
```

```
    MaxPooling2D(pool_size=(2,2)),
    Conv2D(filters=128, kernel_size=(3,3), activation='relu'),
    Flatten(),
    Dense(128, activation='relu'),
    Dropout(0.5),  # Adding dropout for regularization
    Dense(64, activation='relu'),
    Dense(10, activation='softmax')  #using Softmax for multiclass
classification
])

# Compiling model with a tuned learning rate
model.compile(optimizer=Adam(learning_rate=0.0005),
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

# Early stopping to prevent overfitting
early_stopping = EarlyStopping(monitor='val_loss', patience=3,
restore_best_weights=True)

history = model.fit(train_images, y_train, epochs=30,
                    validation_split=0.2, callbacks=[early_stopping])

test_loss, test_accuracy = model.evaluate(test_images, y_test, verbose=0)
print(f"Test Accuracy: {test_accuracy * 100:.2f}%")

model.summary()

import matplotlib.pyplot as plt

plt.figure(figsize=(14,5))
plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.title('Model Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()

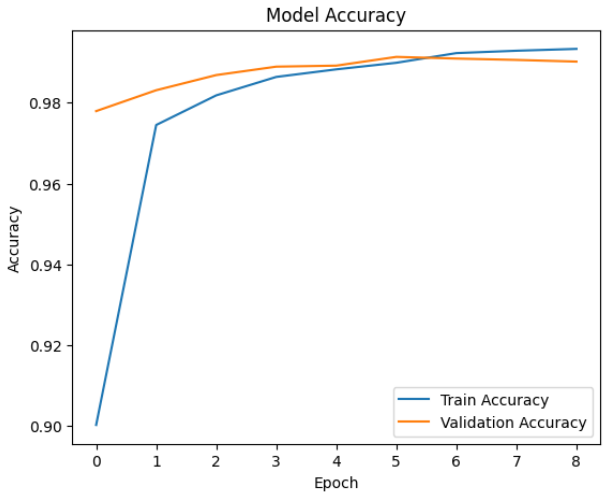
plt.subplot(1, 2, 2)
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title('Model Loss')
plt.xlabel('Epoch')
```

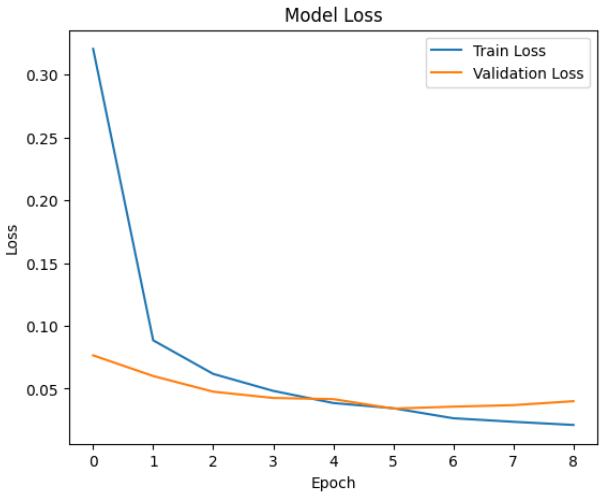
```
plt.ylabel('Loss')
plt.legend()

plt.show()
```

```
1500/1500 [==============================] - 23s 15ms/step - loss: 0.0236 - accuracy: 0.9929 -
Epoch 9/30
1500/1500 [==============================] - 23s 15ms/step - loss: 0.0211 - accuracy: 0.9933 -
Test Accuracy: 99.14%
Model: "sequential_2"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 conv2d_4 (Conv2D)           (None, 26, 26, 32)        320

...
Total params: 249162 (973.29 KB)
Trainable params: 249162 (973.29 KB)
Non-trainable params: 0 (0.00 Byte)
_________________________________________________________________
```





Different Combinations model:

```
from keras.callbacks import EarlyStopping
from keras.optimizers import Adam
import matplotlib.pyplot as plt

# Defining the function to create and train the model
```

```
def train_model(filters, kernel_size, dense_units, dropout_rate,
learning_rate, epochs=10):
    model = Sequential([
        Conv2D(filters=filters, kernel_size=kernel_size,
activation='relu', input_shape=(28,28,1)),
        MaxPooling2D(pool_size=(2,2)),
        Conv2D(filters=filters*2, kernel_size=kernel_size,
activation='relu'),
        MaxPooling2D(pool_size=(2,2)),
        Flatten(),
        Dense(dense_units, activation='relu'),
        Dropout(dropout_rate),
        Dense(dense_units // 2, activation='relu'),
        Dense(10, activation='softmax')
    ])

    model.compile(optimizer=Adam(learning_rate=learning_rate),
                  loss='sparse_categorical_crossentropy',
metrics=['accuracy'])

    early_stopping = EarlyStopping(monitor='val_loss', patience=3,
restore_best_weights=True)

    history = model.fit(train_images, y_train, epochs=epochs,
                  validation_split=0.2, callbacks=[early_stopping],
verbose=0)

    return model, history

# combinations of parameters
combinations = [
    {'filters': 32, 'kernel_size': (3,3), 'dense_units': 128,
'dropout_rate': 0.5, 'learning_rate': 0.0005},
    {'filters': 64, 'kernel_size': (3,3), 'dense_units': 256,
'dropout_rate': 0.3, 'learning_rate': 0.001},
]

results = []
for i, combo in enumerate(combinations):
    model, history = train_model(**combo, epochs=20)
    test_loss, test_accuracy = model.evaluate(test_images, y_test,
verbose=0)
```

```
    results.append({
        'config': combo,
        'model': model,
        'history': history,
        'test_loss': test_loss,
        'test_accuracy': test_accuracy
    })

    # Plot accuracy and loss for each configuration
    plt.figure(figsize=(12, 5))
    plt.subplot(1, 2, 1)
    plt.plot(history.history['accuracy'], label='Train Accuracy')
    plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
    plt.title(f"Configuration {i+1} - Accuracy")
    plt.legend()

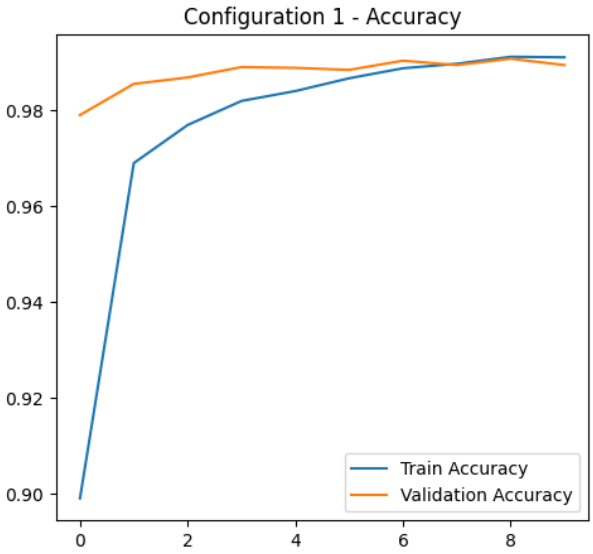
    plt.subplot(1, 2, 2)
    plt.plot(history.history['loss'], label='Train Loss')
    plt.plot(history.history['val_loss'], label='Validation Loss')
    plt.title(f"Configuration {i+1} - Loss")
    plt.legend()

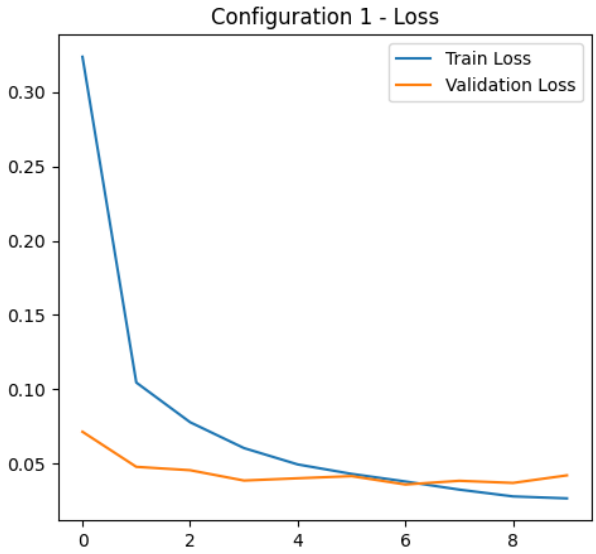
    plt.show()

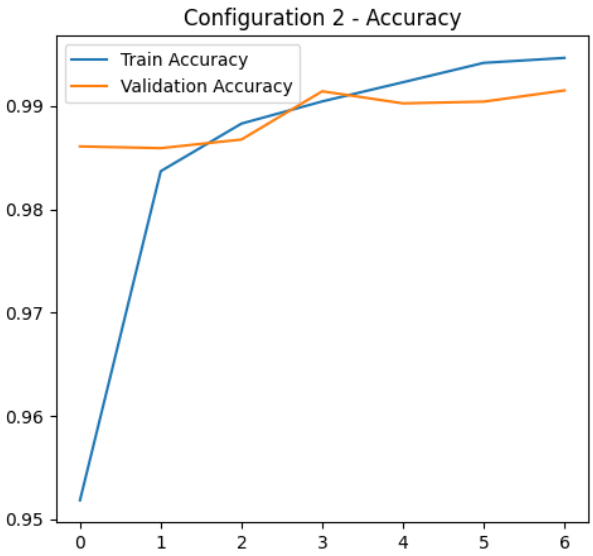


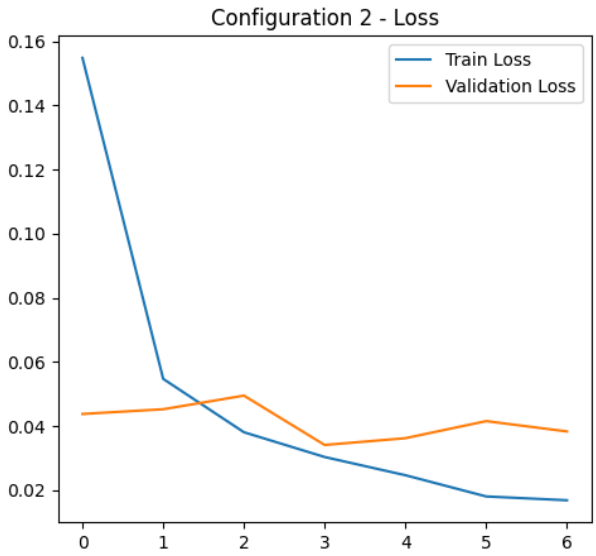
    # Find the best combination based on test accuracy
    best_result = max(results, key=lambda x: x['test_accuracy'])
    best_config = best_result['config']
    best_accuracy = best_result['test_accuracy']

    print("Best Configuration:")
    print(best_config)
    print(f"Test Accuracy: {best_accuracy * 100:.2f}%")
```









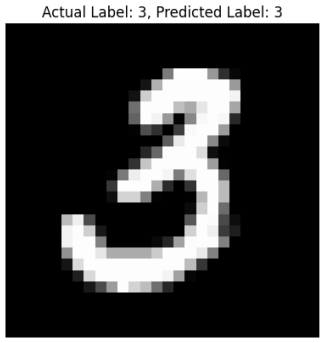

```
Best Configuration:
{'filters': 64, 'kernel_size': (3, 3), 'dense_units': 256, 'dropout_rate': 0.3, 'learning_rate': 0.001}
Test Accuracy: 99.15%
```

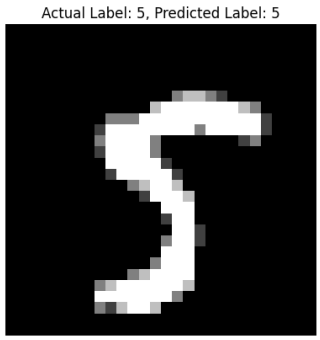
```python
for i in range(10):
    test_index = np.random.randint(0, len(x_test))
    test_image = x_test[test_index]
    test_label = y_test[test_index]

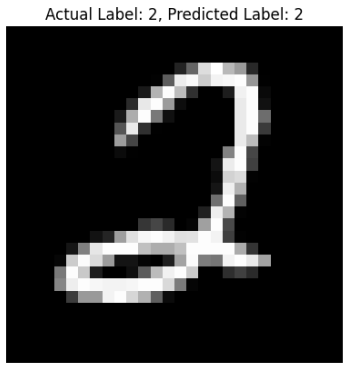
    prediction = model.predict(test_image.reshape(1, 28, 28, 1))
    predicted_label = np.argmax(prediction)
```

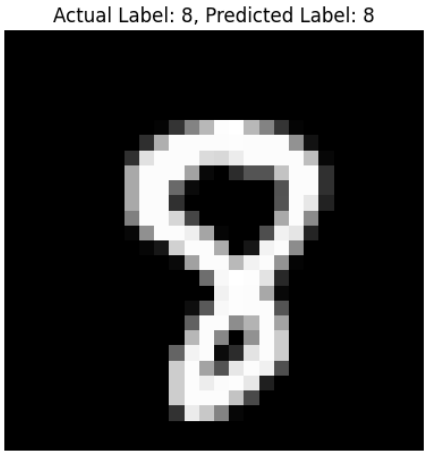
```
    print(f"Image {i+1}:")
    print("Prediction distribution: ", prediction)
    print("Actual Label: ", test_label)
    print("Predicted Label: ", predicted_label)

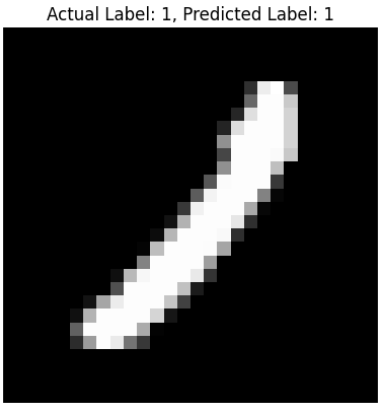
    plt.imshow(test_image.reshape(28, 28), cmap='gray')
    plt.title(f'Actual Label: {test_label}, Predicted Label:
{predicted_label}')
    plt.axis('off')
    plt.show()
```











Analysis:

Analysis of Model Optimization

1. **Architecture Changes**
    - **Original Model**: The initial model was fairly simple, using only two convolutional layers with a small number of filters (6 and 16). It relied on a sigmoid activation function, which, while common in simpler models, often leads to vanishing gradient issues as the network gets deeper. Additionally, it lacked any form of regularization, which could make it prone to overfitting as it scales up.
    - **Optimized Model**: In contrast, the optimized model incorporates several improvements to address the limitations of the original. By switching to ReLU activation, it allows for better gradient flow, making it easier to train deeper networks effectively. The model's depth has also increased, with more convolutional layers and higher filter counts (32, 64, and 128 filters), allowing it

to capture more complex features. Furthermore, dropout regularization was added to reduce overfitting, creating a more robust, modern architecture overall.

2. **Best Configuration Choices**
    - **Kernel Size**: Using a 3x3 kernel size—smaller than the original 5x5—helps detect finer details and improves computational efficiency. This size is a standard in modern CNNs, like VGG and ResNet, due to its effectiveness in feature extraction.
    - **Dense Units**: Increasing the dense layer units to 256 provides the model with greater capacity for feature representation. This added depth in representation power, paired with dropout, helps prevent overfitting while retaining important information.
    - **Dropout Rate**: A dropout rate of 0.3 proved optimal, providing a good balance between regularization and information flow. It's enough to reduce overfitting without cutting off too much useful information.
    - **Learning Rate**: Starting with a learning rate of 0.001, commonly used with the Adam optimizer, enabled stable training with a good balance between convergence speed and stability.
3. **Performance Gains**
    - The optimized model achieved a test accuracy of 99.15%. This success came from multiple factors:
        - ReLU activation, which improved gradient flow.
        - Dropout regularization, effectively reducing overfitting.
        - Increased depth and filters, giving the model sufficient capacity for learning complex features.
        - Careful tuning of hyperparameters, ensuring an ideal balance of speed and stability.
4. **Additional Enhancements**
    - To further improve stability and generalization, early stopping was added, along with a validation split for monitoring training progress. Switching to the Adam optimizer also enhanced the model's efficiency, and learning rate tuning was used to ensure optimal convergence.